

RLLaVA: An RL-central Framework for Language-Vision models and Agents

No Author Given

No Institute Given

Abstract. As vision-language models (VLMs) advance toward more complex reasoning and agentic behaviors, reinforcement learning (RL) has been increasingly applied to multi-modal tasks. However, existing general RL training frameworks do not account for unique characteristics of multi-modal RL, such as multi-modal data transmission and token alignment across multiple images in long contexts. Furthermore, their tight coupling of algorithmic logic and distributed execution is inconvenient for RL researchers with limited GPU resources. To address these challenges, we present an **RL**-central framework for **L**anguage and **V**ision models and **A**gents (**RLLaVA**) with its formulation, natively designed for multi-modal and agentic RL. RLLaVA decouples RL algorithmic logic from model architecture, agentic workflow and distributed execution, facilitates researchers to develop and test new RL algorithms with minimal code while remaining agnostic to specific training and inference engines. RLLaVA makes resource-efficient training of 1B–7B models feasible on common GPUs; notably, 4B-scale models can be trained end-to-end with full-parameter updates on a single 24GB GPU. Experiments on multi-modal and agentic tasks demonstrate the effectiveness and task extensibility of RLLaVA.

Keywords: Reinforcement learning · Vision-language model · Infrastructure · Multi-modal · Agentic.

1 Introduction

Reinforcement learning (RL) has emerged as a pivotal technique for post-training alignment and capacity enhancement of large language models (LLMs), demonstrated by models such as OpenAI o1 [23] and DeepSeek-R1 [4]. Along with these models, a variety of RL training frameworks have emerged such as veRL [35] and OpenRLHF [7]. Recently, RL has been increasingly adopted in multi-modal scenarios, as vision-language models (VLMs) continue to evolve toward more complex reasoning and agentic capabilities, including visual reasoning [15], tool-call [29] and environment interaction [13].

Despite the growing interest of RL in VLMs training, there remains a lack of an natively designed RL training framework for multi-modal agentic RL. Contemporary multi-modal RL research [18, 29] was mainly adapted from general RL frameworks for LLMs [35], which did not take into account the characteristics

inherent in multi-modal agentic RL, such as handling multi-modal observations streamed back from prompts or environments, and aligning visual tokens from multiple images within long contexts.

Moreover, existing general RL frameworks are primarily designed for large-scale clusters and tightly coupling algorithmic logic with distributed execution strategies, making it expensive for researchers to develop and experiment new RL algorithms. This barrier is particularly pronounced for research groups with limited GPU resources.

To address the problems above, we present **RLLaVA**, a lightweight and modular framework specifically designed for multi-modal agentic RL research. We mathematically formulate the joint visual-textual sequential decision of VLMs as a unified Markov decision process (MDP), providing a principled foundation for applying a broad class of RL algorithms to multi-modal and agentic tasks.

In terms of architecture design, RLLaVA adopts a modular and decoupled design that separates RL algorithmic logic, model architecture, agentic workflow, and distributed execution. Inspired by TinyLLaVA Factory [43, 8], RLLaVA further abstracts RL-specific roles into composable components. This design allows researchers to plug in diverse RL algorithms, environments(or tools) and VLMs with minimal changes in code, while remaining agnostic to specific training engines (e.g., FSDP[41], DeepSpeed[27]) and inference engines (e.g., vLLM[11], SGLang[33]).

Additionally, RLLaVA targets researchers and practitioners who conduct resource-efficient training for 1B–7B models on small-scale setups (1–8 GPUs), rather than relying on large-scale clusters. In particular, it supports end-to-end full-parameter training of 4B-scale models on a single 24GB GPU when combined with activation checkpointing and parameter offloading, substantially lowering the entry barrier for multi-modal agentic RL research.

We validate task extensibility of RLLaVA through experiments across diverse multi-modal tasks including geometric reasoning, visual perception (counting, grounding), and multi-modal agentic scenarios (web search, code generation). The results show that the models trained with RLLaVA significantly outperform base models on these tasks, competitive with specially engineered RL frameworks[18].

2 Related Work

2.1 Multi-Modal Language Models

The development of multi-modal language models has progressed from early vision-language understanding [26] to sophisticated architectures capable of complex cross-modal reasoning. LLaVA [17] pioneered effective vision-language integration and established training strategies for multi-modal instruction following. Building upon LLaVA’s foundation, TinyLLaVA Factory [43, 8] demonstrated that well-designed modular frameworks enable small-scale multi-modal models to achieve competitive performance with affordable computational resources. Building on three core principles—lightweight implementation, modular architecture,

and extensibility—TinyLLaVA Factory provides HuggingFace-native interfaces and plug-and-play components with minimal configuration overhead. Our work extends these principles to the reinforcement learning domain, addressing the additional challenges of algorithm diversity and distributed training coordination.

2.2 RL Algorithms and Training Systems

Recent advances in RL for LLMs have introduced diverse algorithms beyond traditional PPO [32], including critic-free group methods (GRPO [34], RLOO [1]), optimal baseline approaches (OPO [14], REMAX [16]), and entropy-preserving methods (CLIP-COV, KL-COV [3]). veRL [35] provides a flexible and comprehensive implementation of multiple RL algorithms with sophisticated distributed training capabilities through its HybridFlow architecture. It introduces a hierarchical hybrid programming model that combines single-controller and multi-controller paradigms, enabling efficient execution of diverse RLHF dataflows. However, veRL is primarily designed for large-scale clusters and requires extensive distributed systems expertise.

Other systems present different trade-offs: OpenRLHF [7] focuses on industrial-scale deployment; AReaL [21] achieves high throughput through asynchronous training but focuses primarily on text-based tasks; RLinf [39] optimizes RL workflows through macro-to-micro flow transformation for flexible execution; ROLL [30] provides unified frameworks for diverse RL modes with large-scale deployments. For multi-modal scenarios, EasyR1 [5] adapts veRL for vision-language models but focuses on selected algorithms.

These systems prioritize scalability and performance optimization for large-scale training, leaving a gap for user-friendly frameworks that extend proven RL implementations to multi-modal scenarios with simplified configuration for resource-constrained research environments.

2.3 Multi-Modal and Agentic RL Applications

With the improvement of large language models (LLMs) and the expansion of their application scenarios, the single-step alignment paradigm represented by RLHF has been extended into an Agentic RL paradigm [40] that involves multi-step, long-horizon sequential decision-making. Recent work has explored agentic RL on various multi-modal tasks such as web search, code generation [18], visual reasoning [29] and gui-agent [13], demonstrating the growing importance of agentic RL in multi-modal domains.

However, these works build upon the above general-purpose RL frameworks which not provide targeted design for multimodal agent scenarios, where the system must handle visual observations, coordinate tool usage, and manage complex multimodal context. RLLaVA bridges this gap by introducing a modular design for the agentic workflow, enabling broader adoption of multi-modal and agentic RL research.

3 RL-Central Framework for Language and Vision Assistants

This paper presents **RLLaVA**, an **RL**-central framework for **L**anguage and **V**ision **A**ssistants, aiming to formulate and implement the agentic RL for VLMs. RLLaVA decouples RL algorithmic logic from model architecture, agentic workflow and distributed execution engines based on the modular method, improving the efficiency of algorithm research teams with limited resources.

3.1 Formulation of RLLaVA

The MDP of RLLaVA The VLMs in RLLaVA are designed to understand visual information and human-like text, and generate human-like text, which could be parsed to executable actions. Thus, it could also be generally formulated as a next token/word prediction problem: given the image $x \in \mathcal{I}$ ¹ and the textual sequence $y_{1:t-1} \in \mathcal{V}$, predict the next token y_t , where $\mathcal{I}$ is the space of images. And RLLaVA can be also viewed as an MDP:

(1) the state space of RLLaVA $\mathcal{S} = (\mathcal{V} \cup \mathcal{I})^*$ (all states consisting of the token from the vocabulary and the image from the image space); (2) the action space of RLLaVA $\mathcal{A} = \mathcal{V}$; (3) the transition distribution of RLLaVA $P(s'|s, a) = 1$ if $s' = (s, a)$, otherwise 0, for arbitrary state s, s' and action a ; (4) the reward function $R((\mathcal{V} \cup \mathcal{I})^*)$ (rule-based or model-based) used to evaluate the actions generated by policy, based on the response of environments. The policy of RLLaVA is itself VLM $\pi_\theta : (\mathcal{V} \cup \mathcal{I})^* \rightarrow \mathcal{V}$. In particular, following the formulation of TinyLLaVA[8], π_θ is estimated by an LLM $\mathcal{F}(\cdot|\theta_1)$, a vision encoder $\mathcal{V}(\cdot|\theta_2)$, and a connector $\mathcal{W}(\cdot|\theta_3)$, where θ consisting of the learnable parameters θ_1, θ_2 and θ_3 .

The Objective of RLLaVA The overall optimization objective of RLLaVA can be expressed as maximizing:

$$\mathbb{E}_{s_0 \sim \mathcal{P}, y_{1:T} \sim \pi_{\theta_{\text{old}}}(\cdot|s_0)} \left[\sum_{t=1}^T L^{PG}(\pi_\theta, \pi_{\theta_{\text{old}}}, \hat{A}_t(s_t, y_t)) - \Omega(\pi_\theta, \pi_{\text{ref}}, s_0, y_{1:T}) \right]. \quad (1)$$

Here, $s_0 = (x_0, y_0)$ is the pair of initial visual and language prompts; $\mathcal{P}$ is the set of task prompts; $y_{1:T}$ are the textual sequences generated as actions, in which T is the number of turns of interaction with the environment; $s_{1:T} \in (\mathcal{V} \cup \mathcal{I})^*$ are the response states of the interaction environment; $\hat{A}_t(s_t, y_t)$ is an estimator of the advantage function to assign suitable credit to states and actions in different time steps based on $R((\mathcal{V} \cup \mathcal{I})^*)$.

In Eq. 1, $L^{PG}(\pi_\theta, \pi_{\theta_{\text{old}}}, \hat{A}_t(s_t, y_t))$ is the policy loss function used to compute the policy gradient. It can be defined differently (e.g., PPO clipping, DAPO

¹ $\mathcal{I}$ denotes the space that contains various images, which can compose new images or videos.

decoupled clipping, GSPO aggregation). If defined as PPO’s clipped surrogate loss, $L^{PG}(\pi_\theta, \pi_{\theta_{\text{old}}}, \hat{A}_t(s_t, y_t)) = \min [r_t(\theta), \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon)] \hat{A}_t$, where $r_t(\theta) = \frac{\pi_\theta(y_t | s_0, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t | s_0, y_{<t})}$ is the likelihood ratio, handling the off-policy distribution mismatch between the behavior policy $\pi_{\theta_{\text{old}}}$ and the target policy π_θ .

$\Omega(\pi_\theta, \pi_{\text{ref}}, s_0, y_{1:T})$ can be various regularization mechanisms (e.g., KL-to-reference, entropy bonuses, n-gram penalties) based on the specific algorithm and training requirements. When instantiated with KL-to-reference regularization, $\Omega(\pi_\theta, \pi_{\text{ref}}, s_0, y_{1:T}) = \beta_{\text{KL}} \sum_{t=1}^T \text{KL}(\pi_\theta(\cdot | s_0, y_{<t}) \| \pi_{\text{ref}}(\cdot | s_0, y_{<t}))$ with $\beta_{\text{KL}} > 0$ controlling the regularization strength.

In the agentic scenarios, consider that pure RL methods may suffers from problems such as sparse rewards, training instability, and low sample utilization efficiency, we introduce the fused SFT-RL unified paradigm, which can utilize process-level supervision signals to mitigate the above difficulties.

Fused SFT-RL recipes (Sec. 4.1) specialize Eq. (1) through segment-wise masks over the response: on-policy spans use the usual policy-gradient contribution L^{PG} with the batch’s advantage estimates, while complementary spans add direct supervised likelihood toward hints or demonstrations, with stability terms still captured by Ω and algorithm-specific choices of π_{ref} .

Above all, the training framework of RLLaVA is shown in Algorithm 1.

Algorithm 1 RLLaVA Training Framework

Input: initial policy $\pi_{\theta_{\text{init}}}$, task prompts $\mathcal{P}$, reward function R , value model V

```

1: policy model  $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$ 
2: for iteration = 1 to  $I$  do
3:   reference model  $\pi_{\text{ref}} \leftarrow \pi_\theta$  if needed
4:   for step = 1 to  $M$  do
5:     Sample a batch of prompts  $\mathcal{P}_b$  from  $\mathcal{P}$ 
6:     Update old policy  $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$ 
7:     Construct rollout inputs with optional SFT-RL strategy, e.g., prefix hints, supervised masks
8:     Generate  $K$  trajectories  $\{\tau^{(k)}\}_{k=1}^K \sim \pi_{\theta_{\text{old}}}(\cdot | s_0)$  by interacting with the environment
9:     Compute rewards  $r^{(k)} = R(\tau^{(k)})$ 
10:    Compute advantages  $\hat{A}_t^{(k)}$  using rewards  $r^{(k)}$  (and using  $V$  if critic-based)
11:    for update iteration = 1 to  $\mu$  do
12:      Update the policy model  $\pi_\theta$  by maximizing the RLLaVA objective (Equation 1)
13:    end for
14:  end for
15: end for
Output:  $\pi_\theta$ 

```

3.2 Architecture Design of RLLaVA

The key design of RLLaVA is to adopt a modular component architecture that encompasses VLM components, RL-specific roles, workflow with environments used for agent loop and engines used for training and inference, enabling flexible composition and selective usage based on algorithm requirements. Figure 1 presents the overall pipeline in RLLaVA, illustrating how the above components participate in the various stages of RL training.

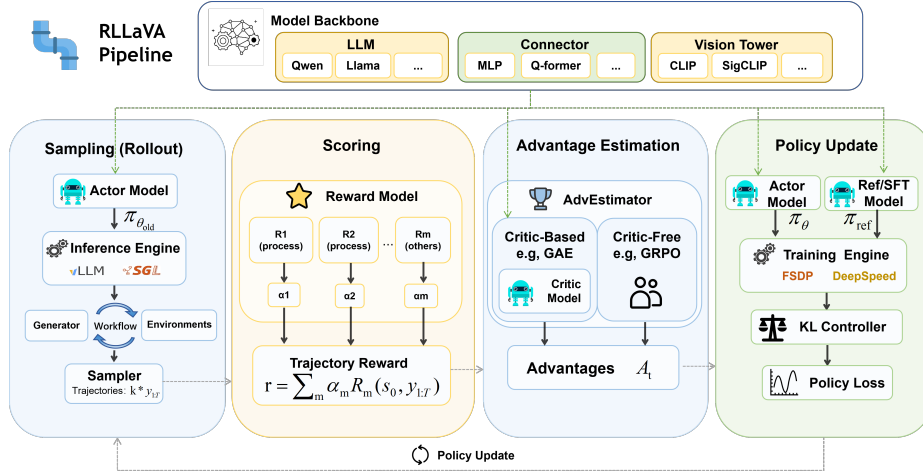


Fig. 1: Overview of RLLaVA pipeline, showing the RL-centric design with four stage: sampling(rollout), scoring, advantage estimation and policy updating.

Decoupled Modules Four Roles of RLLaVA. According to the above RL training paradigm, the framework organizes functionality into four distinct roles, each encapsulates a specific responsibility, and can be independently implemented or configured. **Actor** $\mathcal{R}_{\text{act}}$ manages the policy network to coordinate the policy-environment interaction process and handles policy updates during training. **Reward** $\mathcal{R}_{\text{rew}}$ evaluates the action trajectory, which can be instantiated as rule-based or model-based approaches. **Critic** $\mathcal{R}_{\text{cri}}$ (optional) provides the estimation of the value function for critic-based methods. **Reference** $\mathcal{R}_{\text{ref}}$ maintains a frozen copy of the policy for the KL divergence computation.

Multi-Modal Model Components. Building on TinyLLaVA Factory’s modular design, the framework supports diverse vision encoders (CLIP [26], SigLIP [38], DINOv2 [24]) and language models (Qwen [2], Phi [22]) through factory-based component registration.

Tools and Environments. We develop specific tools (Search API and Code Interpreter) and environments (OSWorld [36], AndroidWorld [28]) for multi-turn interaction of VLM-based agents, enhancing their planing and reasoning capabilities.

Training and Inference Engines. The framework builds upon the PyTorch and HuggingFace Transformers ecosystem, integrating vLLM [11] and SGLang [33] for efficient rollout inference, while providing native training engines (e.g., FSDP/FSDP2 and DeepSpeed) through a unified **TrainEngine** interface.

The role-based method decouples all the components required by the RL framework, facilitating modifications to the algorithm and providing flexibility in algorithmic exploration and hybrid approaches.

Agentic Workflow Agentic RL expands the rollout process from single-shot generation to multi-turn interaction with external tools and environments, including complex context construction for input of models. Therefore, we encapsulate this interaction process into a pluggable workflow, allowing the training mode to switch freely between single-turn and multi-turn interactions. The workflow is illustrated in figure 2.

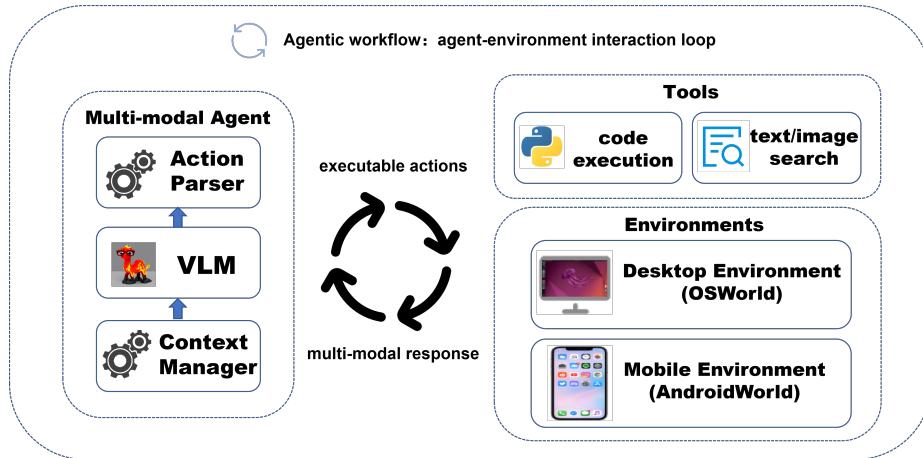


Fig. 2: Agentic workflow of RLLaVA, showing how VLM-based agent interacts with external tools (Code Interpreter and Search API) and environments (OS-World and AndroidWorld).

At each turn T , VLM-based agent generates text base on the current context constructed by context manage system, which encompasses not only the current state but also the interaction history: the original multimodal prompt, model-generated outputs, screenshots and execution feedback from tools or environments. The generated text is no longer the chain-of-thought with answer, but rather the parseable sequence such as '`<think>...</think><action>...</action>`', which may include search query, executable code or GUI actions to be extracted by specific action parser. Then the external tools and environments receive actions and respond multi-modal observations for the next turn. The episode terminates upon reaching the final answer, env done, or max turns. Finally the rewards, advantages, and policy loss are calculated for the complete trajectories.

Training Pipeline Based on the roles introduced in Sec. 3.2 and the workflow in Sec. 3.2, RLLaVA provides a simple pipeline abstraction that orchestrates the standard RL training loop through four key stages, each involving specific roles and additional algorithmic components:

Sampling stage (Role: $\mathcal{R}_{\text{act}}$). $\mathcal{R}_{\text{act}}$ orchestrates the policy-environment interaction by coordinating inference engines (vLLM/SGLang) to execute the policy $\pi_{\theta_{\text{old}}}$ (provided by $\mathcal{R}_{\text{act}}$ at the previous step), and sampling trajectory (a group of trajectories in GRPO) through the interaction with environments (the process of **Rollout**): $y_{1:T} \sim \pi_{\theta_{\text{old}}}(\cdot|s_0)$. This stage embodies the core RL process of gathering experience data for policy improvement.

Scoring stage (Role: $\mathcal{R}_{\text{rew}}$). This stage evaluates the collected trajectories by computing scalar rewards that reflect the quality of each whole trajectory. $\mathcal{R}_{\text{rew}}$ processes each trajectory $y_{1:T}$ and computes a combined reward signal: $r = \sum_m \alpha_m R_m(s_0, y_{1:T})$, where each R_m represents a reward metric (potentially from different sources, including both outcome rewards and process rewards) weighted by α_m . These rewards serve as the foundation for advantage estimation stage and policy update stage.

Advantage estimation stage (Role: $\mathcal{R}_{\text{cri}}$, Algorithmic component: AdvEstimator). This stage computes advantage estimates that quantify the relative value of actions in a trajectory. A pluggable AdvEstimator processes rewards to compute advantages: $A_t \leftarrow \text{AdvEstimator}(r)$, where different algorithms may incorporate additional information and implement different strategies as needed: critic-free methods (e.g., GRPO) compute advantages directly from rewards, while critic-based methods (e.g., GAE) additionally leverage value estimates from the critic role $\mathcal{R}_{\text{cri}}$.

Policy update stage (Roles: $\mathcal{R}_{\text{act}}$, $\mathcal{R}_{\text{ref}}$, $\mathcal{R}_{\text{cri}}$, Algorithmic components: PolicyLoss, KLController). The actor role $\mathcal{R}_{\text{act}}$ and reference role $\mathcal{R}_{\text{ref}}$ jointly compute regularization terms (e.g., $\text{KLController}(\pi_{\theta}(\cdot|s_0, y_{<t}) \parallel \pi_{\text{ref}}(\cdot|s_0, y_{<t}))$), where π_{θ} comes from $\mathcal{R}_{\text{act}}$ and π_{ref} from $\mathcal{R}_{\text{ref}}$. The pluggable policy loss function $\text{PolicyLoss}(\pi_{\theta}, \pi_{\text{old}}, \hat{A}_t(s_t, y_t))$ computes the policy gradient, which is combined with regularization to form the complete loss.

Different algorithms implement different policy loss strategies. The actor role $\mathcal{R}_{\text{act}}$ and $\mathcal{R}_{\text{cri}}$ (if critic-based method used) updates the policy parameters based on the loss.

With the support of this flexible and extensive training pipeline, researchers could focus on algorithmic innovation without dealing with system complexity.

4 Implementation

Building upon the framework described above, this section describes the implementation of RLLaVA, focusing on how algorithm plugins, agent-environment runtime, and execution backends are instantiated in the codebase.

4.1 Algorithm Plugin Implementation

The algorithm plugin system is implemented through a registration-based architecture inspired by veRL, enabling rapid algorithm integration and experimentation. Users can implement custom algorithms by writing functions that define the core algorithmic logic, and register them as pluggable components. In RLLaVA, algorithm extensibility mainly occurs at two points: advantage estimation (e.g., GAE[31], GRPO[34], RLOO[1], REMAX[16], OPO[14], REINFORCE++[6]) and policy loss variants (e.g., vanilla, GSPO[42], CLIP-COV and KL-COV[3]), which can be selected through configuration without touching the training loop. The same interface extends to SFT-RL fusion methods like [20, 37], where policy-gradient updates are paired with auxiliary supervised-likelihood terms on ground-truth or reference responses, hints, or other annotated response spans. This mitigates sparse rewards and unreliable early rollouts while matching the template of Eq. (1). Concretely, trajectory annotation after rollout records marks on tokens or groups and may replace weak responses with references, while a chosen policy-loss plugin consumes these marks together with advantage estimates and Ω . Both stages are toggled through configuration, so fused training reuses the existing four-stage loop without a separate trainer.

The system currently integrates a comprehensive collection of RL components and commonly used variants. For example, it includes GRPO’s group-based normalization, RLOO’s leave-one-out baselines, OPO’s length-weighted baselines, DAPO’s decoupled clip ratios, and GSPO’s sequence-level aggregation. The composable design further allows mixing and matching components (e.g., OPO baseline with CLIP-COV selective updates, or RLOO advantage with GSPO loss aggregation), enabling flexible algorithmic exploration and hybrid approaches.

4.2 Agentic Context Management and Isolated Environment Runtime

Token-in-token-out context management. A central implementation challenge in multi-modal agentic RL is context manage for generation and the training-side likelihood computation alignment. Although the approach of decoding response tokens into natural language text and then reconstructing the context is simple and convenient, it suffers from the tokenization shift problem. This problem arises because some existing tokenizers do not guarantee encode-decode consistency, i.e., $Encoder(Decoder(token_ids)) \neq token_ids$, which leads to computing log probabilities over an incorrect distribution during RL and consequently causes training collapse. Therefore the context manager of RLLaVA constructs context in a **token-in-token-out** manner: policy-generated assistant turns are appended using the exact sampled token ids returned by the rollout engine, while tool and environment observations are appended as template deltas. This token-in-token-out design makes the sampled trajectory the source of truth for both subsequent generation and policy-gradient computation.

In addition, different context management strategies can be introduced, such as sliding-window truncation, history compression, or long-term agent memory, by replacing the context policy without changing the RL training loop.

Subprocess-based environment runtime. For heavyweight interactive benchmarks such as desktop or mobile GUI environments, RLLaVA isolates the actual environment execution in subprocess workers. The main training process communicates with each worker through a lightweight JSON protocol for reset, step, evaluate, and close commands. This design decouples the RL training process from environment-specific dependencies, virtual machines or emulator lifecycle management, convenient for rollout in parallel environments. The environment-side observation is converted into a unified environment response, including screenshots, textual instructions, step rewards, and termination signals, which is then injected back into the multi-turn context.

4.3 Distributed Execution Strategies and Resource-Efficient Training.

RLLaVA provides native inference engines (vLLM[11], SGLang[33], HuggingFace) and training engines (FSDP/FSDP2[41] and DeepSpeed[27]), supporting stage-wise weight synchronization and memory management in the co-located rollout-training workflow.

RLLaVA targets resource-constrained setups and provides practical strategies to reduce memory overhead across the RL training pipeline. In practice, its memory efficiency mainly benefits from two mechanisms. First, RLLaVA adopts a co-located execution strategy that multiplexes GPU memory between rollout and optimization stages: during rollout the training engine keeps model states largely CPU-offloaded, while during optimization the rollout inference engine enters a sleep/offload mode, so that training and inference do not concurrently occupy GPU memory. Second, RLLaVA leverages sharded training with CPU offload (e.g., the FSDP2 offload policy or DeepSpeed ZeRO-Offload), so that model states do not need to be fully resident on a single GPU; instead, parameters are materialized on-demand for the active modules and can be offloaded across devices/host memory during optimization.

By default, RLLaVA enables the FSDP2 offload policy in its training configuration, making most integrated tasks feasible on a single 24GB GPU. For example, with Qwen3-VL-4B on counting and math tasks, we observe peak GPU memory of ~ 21 GB during vLLM rollout and ~ 21 – 22 GB during optimization under the co-located execution strategy.

5 Experiments

To validate the effectiveness and task extensibility of RLLaVA, we performed experiments on various multimodal and agentic tasks to evaluate the performance of models trained with RLLaVA, as well as their generalization capability on certain tasks.

5.1 Experimental Setup

Tasks and Datasets. We assess the framework across five multi-modal tasks: (1) **Math**: Geometry3K [19] for diagram-based geometric reasoning, (2) **Counting**: CLEVR-Count-70k [9] for compositional object counting, (3) **Grounding**: RefCOCO/+g [10] for referring expression comprehension (evaluated on both in-domain test sets and out-of-domain LISA benchmark [12]), (4) **Search**: MAT-Search [18] for visual information retrieval, and (5) **Coding**: MAT-Coding [18] for image manipulation with code generation.

Models and Training Configuration. We choose Qwen2-VL-2B and Qwen2.5-VL-3B[25] as our base models. All experiments use GRPO as the primary algorithm for full-parameter fine-tuning, FSDP for distributed training, and vLLM for rollout generation.

5.2 Evaluation Results Across Downstream Tasks

We first evaluate the framework’s effectiveness across diverse multi-modal single-turn tasks, demonstrating its versatility and the consistent benefits of RL fine-tuning.

Table 1: Performance across five multi-modal single-turn tasks. RL fine-tuning with GRPO consistently improves over base models across all domains.

Task	Model	Dataset	Metric	Base	GRPO
Math	Qwen2.5-VL-3B	Geometry3K	Accuracy	35.1	39.0
Counting	Qwen2.5-VL-3B	CLEVR-Count	Accuracy	52.0	57.5
Grounding	Qwen2-VL-2B	RefCOCO/+g	IoU	51.3	63.3
Search	Qwen2.5-VL-3B	MAT-Search	F1	6.0	16.6
Coding	Qwen2.5-VL-3B	MAT-Coding	F1	16.5	28.1

Table 1 shows that RL fine-tuning consistently improves performance across all tasks. Specifically, tool-call tasks (Search, Coding) show the most dramatic improvements (+22.7 and +13.7 F1 points), while perception tasks (Counting, Grounding) also benefit substantially (+5.5 and +12.0 points). These results demonstrate the framework’s versatility in handling diverse multi-modal RL scenarios with a unified training pipeline.

5.3 Generalization Analysis: Visual Grounding

To assess generalization capability of RL, we evaluate the grounding task on both in-domain and out-of-domain benchmarks. The results are presented in Table 2.

RL fine-tuning substantially improves grounding accuracy across both in-domain (RefCOCO/+g) and out-of-domain (LISA) benchmarks. The significant improvement on LISA (+11.10 IoU) is particularly noteworthy, as this dataset requires reasoning-based grounding beyond direct visual-text alignment. This

Table 2: Visual grounding performance on in-domain (RefCOCO/+ /g) and out-of-domain (LISA) benchmarks. RL training improves both task-specific performance and cross-domain generalization.

Model	RefCOCO	RefCOCO+	RefCOCOg	LISA
Base (Qwen2-VL-2B)	54.79	51.48	56.75	20.78
GRPO (300 steps)	67.14	60.43	61.43	31.88
Improvement	+12.35	+8.95	+4.68	+11.10

demonstrates that RL training enhances semantic understanding rather than domain-specific memorization.

5.4 Performance on Multi-Modal Agentic Tasks

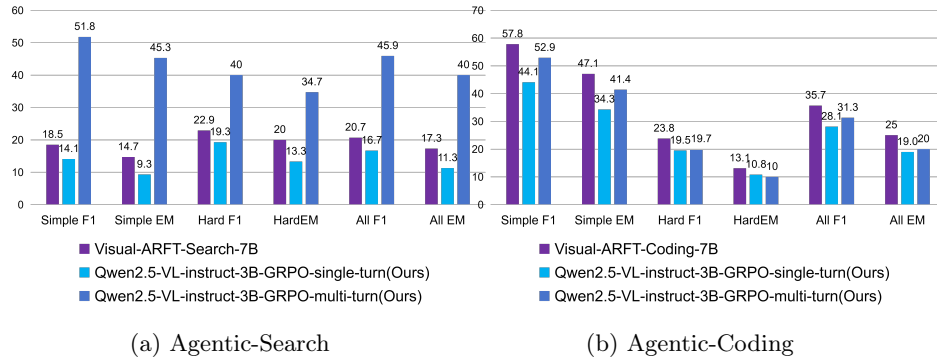


Fig. 3: Performance comparison on agentic-search (left) and agentic-coding (right) tasks. The models trained through our framework achieve competitive or superior performance compared to the models from Visual-ARFT [18].

When it comes to multi-turn tasks, we evaluate on two scenarios of thinking with images (Agentic-Search and Agentic Code), which both require multi-turn planning and tool use. The results are shown in Figure 3.

On the agentic-search task, the 3B model trained through RLLaVA surpasses the 7B Visual-ARFT-Searching [18] clearly (+29.2% All F1) while using the agentic training pipeline. And on agentic-coding task, our 3B model also achieves competitive performance compared with the 7B Visual-ARFT-Coding [18].

These results demonstrate that RLLaVA’s general-purpose framework can match or approach task-specific implementations while offering significantly greater flexibility and extensibility.

6 Conclusion

We have presented RLLaVA, a RL-driven modular framework for multi-modal agentic reinforcement learning research with its formulation. The framework decouples algorithm logic from model architecture, agentic workflow and distributed execution, with a collection of RL algorithms, native multi-modal support, and resource-efficient training strategies for small-scale setups, facilitating resource-constrained research teams. Experiments demonstrate the framework’s effectiveness across tasks in multi-modal agentic scenarios, while offering greater flexibility and extensibility.

Acknowledgments. This work was partially supported by the National Science and Technology Major Project (2022ZD0116310), National Natural Science Foundation of China (Grant No. 62476016 and 62441617), the Fundamental Research Funds for the Central Universities. This work utilized generative AI tools for drafting some text and code segments.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Ahmadian, A., Cremer, C., Gallé, M., Fadaee, M., Kreutzer, J., Pietquin, O., Suleman, K., Pal, C.J., Sordoni, A.: Back to basics: Revisiting reinforce-style optimization for learning from human feedback in llms. arXiv preprint arXiv:2402.14740 (2024), introduces RLOO (Reinforcement Learning with Leave-One-Out)
2. Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., et al.: Qwen technical report. arXiv preprint arXiv:2309.16609 (2023)
3. Cui, G., et al.: Entropy-preserving methods with covariance updates. Algorithm from verl framework (2025), cLIP-COV and KL-COV methods, specific arxiv reference to be confirmed
4. DeepSeek-AI, et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint (2025)
5. EasyR1 Team: Easyr1: A clean multimodal rl framework. GitHub Repository (2024)
6. Hu, J.: Reinforce++: A simple and efficient approach for aligning large language models. arXiv preprint arXiv:2501.03262 (2025)
7. Hu, J., Wu, X., Zhu, Z., Xianyu, Wang, W., Zhang, D., Cao, Y.: Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. arXiv preprint arXiv:2405.11143 (2024)
8. Jia, J., Hu, Y., Weng, X., Zhou, B., Luo, J., Liu, X., Wu, J., Huang, L.: Tinyllava factory: A modularized codebase for small-scale large multimodal models. arXiv preprint arXiv:2405.11788 (2024)
9. Johnson, J., Hariharan, B., Van Der Maaten, L., Fei-Fei, L., Lawrence Zitnick, C., Girshick, R.: Clevr: A diagnostic dataset for compositional language and elementary visual reasoning. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp. 2901–2910 (2017)
10. Kazemzadeh, S., Ordonez, V., Matten, M., Berg, T.: Referitgame: Referring to objects in photographs of natural scenes. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 787–798 (2014)

11. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with pagedattention. In: Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (2023)
12. Lai, X., Tian, Z., Chen, Y., Li, Y., Yuan, Y., Liu, S., Jia, J.: Lisa: Reasoning segmentation via large language model. arXiv preprint arXiv:2308.00692 (2023)
13. Li, P., Hu, Z., Shang, Z., Wu, J., Liu, Y., Liu, H., Gao, Z., Shi, C., Zhang, B., Zhang, Z., et al.: Efficient multi-turn rl for gui agents via decoupled training and adaptive data curation. arXiv preprint arXiv:2509.23866 (2025)
14. Li, Y., et al.: Opo: Optimal policy optimization for language model alignment. Technical Report (2025), algorithm details from research community, specific arxiv reference to be confirmed
15. Li, Y., et al.: Visual reasoning with reinforcement learning. Research in progress (2024), referenced in related work, specific arxiv to be confirmed
16. Li, Z., Xu, T., Zhang, Y., Lin, Z., Yu, Y., Sun, R., Luo, Z.Q.: Remax: A simple, effective, and efficient reinforcement learning method for aligning large language models. arXiv preprint arXiv:2310.10505 (2023)
17. Liu, H., Li, C., Wu, Q., Lee, Y.J.: Visual instruction tuning. *Advances in neural information processing systems* **36**, 34892–34916 (2023)
18. Liu, Z., Sun, Z., Zang, Y., Dong, X., Cao, Y., Duan, H., Lin, D., Wang, J.: Visual-art: Visual agentic reinforcement fine-tuning. arXiv preprint arXiv:2505.14246 (2025), includes MAT-Search and MAT-Coding benchmarks for multi-modal agentic tasks
19. Lu, P., Gong, R., Jiang, S., Qiu, L., Huang, S., Liang, X., Zhu, S.C.: Inter-gps: Interpretable geometry problem solving with formal language and symbolic reasoning. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). pp. 6774–6786 (2021)
20. Lv, X., Zuo, Y., Sun, Y., Liu, H., Wei, Y., Chen, Z., Zhu, X., Zhang, K., Wang, B., Ding, N., Zhou, B.: Towards a unified view of large language model post-training. arXiv preprint arXiv:2509.04419 (2025), introduces the Unified Policy Gradient Estimator (UPGE) and Hybrid Post-Training (HPT)
21. Mei, K., Pang, B., Xiong, C., Yu, D.: Areal: A large-scale asynchronous reinforcement learning system for language reasoning. arXiv preprint arXiv:2501.07906 (2024)
22. Microsoft Research: Phi-2: The surprising power of small language models. Microsoft Research Blog (2023)
23. OpenAI: Openai o1 model (2024)
24. Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., et al.: Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research* (2023)
25. Qwen Team: Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. arXiv preprint arXiv:2409.12191 (2024)
26. Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al.: Learning transferable visual models from natural language supervision. In: International conference on machine learning. pp. 8748–8763. PMLR (2021)
27. Rasley, J., Rajbhandari, S., Ruwase, O., He, Y.: Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 3505–3506 (2020)

28. Rawles, C., Clinckemallie, S., Chang, Y., Waltz, J., Lau, G., Fair, M., Li, A., Bishop, W., Li, W., Campbell-Ajala, F., et al.: Androidworld: A dynamic benchmarking environment for autonomous agents. In: International Conference on Learning Representations. vol. 2025, pp. 406–441 (2025)
29. Ren, Y., Yang, Y., Chen, Y., Li, G., Zhang, J., Yang, H., Ma, X., Jia, X., Liang, F.: Deepeyes: Incentivizing "thinking with images" via reinforcement learning. arXiv preprint arXiv:2505.14362 (2025)
30. ROLL Team, other contributors: Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library. arXiv preprint arXiv:2505.24298 (2024)
31. Schulman, J., Moritz, P., Levine, S., Jordan, M., Abbeel, P.: High-dimensional continuous control using generalized advantage estimation (2018), <https://arxiv.org/abs/1506.02438>
32. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 (2017)
33. SGLang Team: Sglang: Efficient execution of structured language model programs. GitHub Repository (2024), <https://github.com/sgl-project/sglang>
34. Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., Guo, D.: Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300 (2024), introduces GRPO (Group Relative Policy Optimization)
35. Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., Wu, C.: Hybridflow: A flexible and efficient rlhf framework. arXiv preprint arXiv:2409.19256 (2024), open-sourced as verl, accepted by EuroSys 2025
36. Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T.J., Cheng, Z., Shin, D., Lei, F., et al.: Osvorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. Advances in Neural Information Processing Systems **37**, 52040–52094 (2024)
37. Yan, J., Li, Y., Hu, Z., Wang, Z., Cui, G., Qu, X., Cheng, Y., Zhang, Y.: Learning to reason under off-policy guidance. arXiv preprint arXiv:2504.14945 (2025), accepted at NeurIPS 2025
38. Zhai, X., Mustafa, B., Kolesnikov, A., Beyer, L.: Sigmoid loss for language image pre-training. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 11975–11986 (2023)
39. Zhang, B., Liu, Y., Wang, S., Gao, C., Cao, J., Xu, S., Dong, X., Zhao, K., et al.: Rlinf: Flexible and efficient large-scale reinforcement learning via macro-to-micro flow transformation. arXiv preprint arXiv:2509.15965 (2024)
40. Zhang, G., Geng, H., Yu, X., Yin, Z., Zhang, Z., Tan, Z., Zhou, H., Li, Z., Xue, X., Li, Y., et al.: The landscape of agentic reinforcement learning for llms: A survey. arXiv preprint arXiv:2509.02547 (2025)
41. Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., Desmaison, A., Balioglu, C., Damania, P., Nguyen, B., Chauhan, G., Hao, Y., Mathews, A., Li, S.: Pytorch fsdp: Experiences on scaling fully sharded data parallel (2023), <https://arxiv.org/abs/2304.11277>
42. Zheng, C., Liu, S., Li, M., Chen, X.H., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., Zhou, J., Lin, J.: Group sequence policy optimization (2025), <https://arxiv.org/abs/2507.18071>
43. Zhou, B., Hu, Y., Weng, X., Jia, J., Luo, J., Liu, X., Wu, J., Huang, L.: Tinyllava: A framework of small-scale large multimodal models. arXiv preprint arXiv:2402.14289 (2024)